

UdaanHer Saathi, Reframed: A Personal AI Mentor

Research, a reality check, and a two-feature MVP plan for the competition

Student: Aishi • Plan written 9 July 2026

1. What changed, and why this document exists

The original brief describes a *kiosk* that walks a woman through twelve steps toward job readiness. This document deliberately puts the kiosk aside. The kiosk is hardware and deployment — it comes later. What we are actually building first is the thing *inside* the kiosk: a personal teacher-mentor, an AI agent that talks with one woman in her own language, gets to know her, teaches her a vocational skill one lesson at a time, checks her understanding through conversation, and remembers everything so the next session picks up where the last one ended.

That reframe matters because it changes what “done” looks like. A kiosk project is done when the box works. A mentor project is done when one woman can sit through one complete teaching loop — *know me, teach me, check me, re-teach me where I am stuck* — and it feels like talking to a patient didi, not filling a form. Everything in this plan is aimed at that loop.

2. Start from her side of the screen

Before features, hold one person in mind. Call her Sunita, 32, from a village near Nadiad. She stitches blouses for neighbours and wants to earn properly — maybe tailoring as a real trade, maybe beauty work, she is not sure. She speaks Gujarati, understands Hindi, reads very little. She has held a smartphone (her husband’s) but has never installed an app. Nobody has ever asked her, in her own language, “what are you already good at?”

Now walk her through our app and notice what she never does:

- She never types. The agent speaks first, she speaks back. Buttons exist, but they are big, few, and optional.
- She never navigates. There is no menu she must understand. The *agent* decides what appears on screen — when it asks about her interests, interest cards appear; when it teaches, the lesson visual appears. The software follows the conversation, not the other way round.
- She never takes a test that looks like a test. The skill assessment is the agent asking, “Have you ever stitched a fall on a saree? Tell me how you do it.” Her answer, in her own words, tells the agent more than any quiz. The same applies to the viva after a lesson: it is a chat, and where she stumbles, the agent circles back and teaches that one point again, right there, with a picture or a video.
- She never wonders if the machine remembers her. Session two opens with “Namaste Sunita! Last time we finished blouse-neck cutting. Shall we see if it stuck?”

This “she never...” list is the real spec. Any feature that forces her to type, navigate, or feel examined is a bug against the spec, however impressive it looks in a demo.

3. A reality check on the original brief

The brief is a good vision and an impossible first build. Ten features, twelve journey steps, kiosk hardware, platform-specific training, mock interviews, simulations, a readiness score. For a competition MVP, most of it must wait. Specifically:

- **Facial-scan login: postpone, and think hard before ever building it.** Biometric data of a vulnerable population is the most sensitive data class under India’s DPDP Act 2023. For the MVP, a name plus a spoken 4-digit phrase (or a facilitator-issued card number) logs her in. Face login adds legal risk and zero demo value right now.
- **The marketplace (Urban Company hiring): postpone.** A marketplace is a second product with its own users (employers), trust problems, and verification burden. What we *keep* from this idea is the data shape: the agent’s progress records are designed so that one day they can render as a verifiable skill profile. We store it; we do not build the storefront.
- **“Employment readiness score”: soften it.** A single AI-decided number scoring a human being is hard to defend and easy to attack in judging Q&A. Replace it with a transparent skill checklist: lessons completed, concepts demonstrated in viva, concepts still shaky. Same information, honest form.
- **Five languages: design for five, ship two or three.** The architecture supports any language Sarvam supports (Section 5); the demo needs Gujarati or Hindi plus English to prove the point. Adding a language later is configuration, not engineering.
- **What survives, at full strength:** the voice-first multilingual mentor, the get-to-know-her onboarding, personalised lesson generation, the conversational viva with re-teaching, and persistent progress. That is the product.

4. Web app or mobile app? Web — specifically a mobile-first PWA

Start with a **web app built mobile-first**, and treat “mobile app” as a packaging step for later. The reasons, in order of weight:

1. **Iteration speed.** You are two people on a competition timeline. A web app deploys in seconds to a URL; a mobile app means emulators, signing, and (eventually) Play Store review. Every hour spent on Android plumbing is an hour not spent on the teaching loop.
2. **The browser already has everything the MVP needs.** Microphone via `MediaRecorder`, audio playback, camera (if ever needed) — all standard web APIs. Sarvam and Claude are called from your backend anyway, so the client is thin.
3. **It demos anywhere.** At the competition you open a URL on a laptop, a judge’s phone, or a borrowed tablet. That flexibility is worth a lot on demo day.
4. **It *is* the kiosk.** The eventual kiosk is realistically a cheap Android tablet running a browser in fullscreen mode. Building the web app first means the kiosk later costs almost nothing.
5. **The path to mobile stays open.** A PWA (progressive web app) can be “installed” to a phone home screen today, and the same codebase wraps into a real Android app with Capacitor if you ever need Play Store presence or better offline support.

The one honest disadvantage of web: weaker offline capability and no reliable background audio. Neither matters for an MVP demonstrated on good Wi-Fi. Revisit when real village deployment (patchy 4G) becomes the problem — that is a post-competition problem.

5. Languages and Sarvam: what is actually possible

The question was: which five languages, including English, do Sarvam’s models support? The answer depends on which model is the bottleneck, because our agent needs the *full voice loop* — hear her (speech-to-text), think, and speak back (text-to-speech):

Sarvam model	Job in our system	Language coverage
Saaras v3 (STT)	Hears Sunita; also handles code-mixed speech (Gujarati sentences with Hindi/English words)	22 Indian languages + English
Bulbul v3 (TTS)	The mentor’s voice; 30+ speaker voices, so we pick a warm female voice	10 Indian languages + English (the bottleneck)
Mayura / Sarvam-Translate	Translating lesson content and YouTube-video summaries into her language	11 / 22 languages + English
Sarvam-30B (LLM) / 105B	Optional secondary brain for Indic phrasing	Indic-tuned chat models

Because the mentor must *speak*, Bulbul’s list is the constraint. Its eleven languages are: **Hindi, Bengali, Tamil, Telugu, Gujarati, Kannada, Malayalam, Marathi, Punjabi, Odia, and English**. Any five of these work end-to-end.

Recommended five: English, Hindi, Gujarati, Marathi, and Bengali. English and Hindi are non-negotiable (brief requirement plus Hindi as the fallback language for teaching videos). Gujarati and Marathi cover western India where a pilot is most plausible for you. Bengali adds the second-largest speaker base; swap it for Tamil or Telugu the day a southern pilot appears — it is a configuration change, one line per language.

Two practical notes. First, Sarvam gives Rs. 100 of free credits on signup and runs a startup programme with 6–12 months of free credits — apply early; a competition project is exactly what it is for. Second, keep Claude as the reasoning brain and Sarvam as ears, mouth, and translator; Sarvam’s LLMs are a fallback, not the core.

6. Do we need to train a model? Do we need a dataset?

No custom model for the MVP — and probably not for a long time. The temptation to “train our own model” is worth resisting explicitly, because judges may ask:

- The hard AI problems here — understanding accented Gujarati speech, generating natural Indic speech — are already solved by Sarvam, trained on more Indic data than we could ever collect.
- The reasoning problems — assessing skill from a story, planning a lesson, running a gentle viva — are exactly what frontier LLMs (Claude / GPT) do well when given a good harness. Fine-tuning would make them worse, not better, at this stage.

- Our defensible asset is not a model. It is the *pedagogy encoded in the agent* plus, over time, the interaction data.

But yes, we need datasets — three small, curated ones, built by hand:

1. **A skill curriculum library.** For one or two skills (start with tailoring), a structured syllabus: modules, lessons, the 3–5 concepts each lesson must land, and common mistakes. Seed it from NSDC/PMKVY Qualification Packs, which already define vocational competencies for tailoring, beauty, and childcare — free, government-standard grounding. This is a JSON/YAML file the agent reads; the agent *personalises* lessons from it rather than inventing curricula from nothing (which is where hallucination would creep in).
2. **An assessment rubric per lesson.** For each concept: what a correct explanation sounds like from a non-literate learner (spoken, informal, in her words), and what common misconceptions sound like. This is what makes the viva fair and the re-teaching targeted.
3. **A curated visual-aid index.** A hand-picked list of good Hindi/Gujarati YouTube tutorials and simple diagrams, tagged by concept. Curated beats searched: the agent picking from 30 vetted links is reliable; live-searching YouTube in a demo is a coin flip.

Later, with consent, logged conversations become an evaluation set (“did the viva catch the gap?”) — that is when DSPy gets interesting (next section). No model training anywhere in this plan.

7. Architecture: the harness, and how the agent drives the screen

7.1 The voice loop

One turn of conversation flows like this:

1. Browser records her speech and sends audio to our backend (FastAPI, Python).
2. **Sarvam Saaras v3** transcribes it (handling code-mixing) into text.
3. The **LangGraph agent**, with **Claude** as its brain, reads the transcript plus her profile and decides two things: *what to say* and *what the screen should show*.
4. The reply text goes to **Sarvam Bulbul v3**, which returns audio in her language; the browser plays it while the screen updates.

7.2 “The agent controls the software” — concretely

This was a key requirement, and it has a clean implementation: **the agent’s tool calls are the UI**. The frontend is a thin renderer with no navigation of its own. Alongside every spoken reply, the agent emits one structured UI command, and the frontend just renders whatever arrives:

Agent tool call	What Sunita sees
<code>show_options(cards)</code>	Big picture-cards (tailoring, beauty, cooking) she can tap <i>or</i> answer aloud
<code>show_lesson(step, image)</code>	One lesson step: a diagram or photo, one line of large text, agent narrating
<code>show_video(url)</code>	An embedded curated video in her language (or Hindi fallback)
<code>show_progress(profile)</code>	Her garden/ladder view: lessons done, concepts strong, concepts to revisit
<code>update_profile(facts)</code>	Nothing visible — the agent writing down what it learned about her

This pattern (sometimes called generative or agent-driven UI) is also your best demo moment: judges visibly watch the AI steer the application in real time.

7.3 The agent itself: LangGraph now, DSPy later

Model the mentor in **LangGraph** as a small state machine whose states mirror the pedagogy: `greet` → `discover` (interests, background) → `assess` (informal skill stories) → `teach` (one lesson, step by step) → `viva` (conversational check) → `reteach` (loop back on shaky concepts) → `close` (save progress, preview next session). LangGraph is the right choice precisely because it gives you *control*: the graph guarantees the agent cannot skip assessment or wander off-curriculum, while Claude supplies the warmth and judgement inside each state. Its built-in checkpointing also gives you session memory nearly for free.

DSPy is a later optimisation, not an MVP component. Its value arrives once you have logged real conversations: then you can compile better prompts for, say, the viva-grading step against actual examples. Mentioning this in your competition Q&A (“our roadmap uses DSPy to optimise assessment prompts against field data”) is stronger than bolting it on now.

Storage is deliberately boring: SQLite (upgradeable to Postgres) holding her profile, skill graph, lesson history, and per-concept mastery. Claude/GPT credits handle reasoning; Sarvam credits handle every audio second; the harness is the FastAPI layer that glues them and enforces the graph.

8. The MVP: two strong features, cleanly cut

8.1 Feature 1 — “The mentor gets to know her”: voice onboarding + skill discovery

One continuous conversation, 5–7 minutes, fully in Gujarati or Hindi (plus English), that produces a persistent learner profile.

- Agent greets her by voice, asks her name, where she is from, what work she already does.
- Interest discovery with `show_options` cards — she taps or just says it.
- Informal skill assessment: 3–4 story-questions about the chosen skill (“how would you take a blouse measurement?”), from which Claude infers a starting level — explicitly *not* a quiz.
- Output on screen: her profile card, read aloud back to her for confirmation (“Did I understand you right, Sunita?”), then saved. Returning users are greeted by name with their history.

Why this feature first: it proves the three hardest claims at once — real Indic voice loop, agent-driven UI, and assessment without literacy. It is also the foundation every other feature reads from.

8.2 Feature 2 — “The mentor teaches and checks”: one lesson + conversational viva + re-teach

The complete teaching loop for *one* lesson of *one* skill (tailoring, from the curated curriculum).

- The agent picks the right lesson from the curriculum library given her profile, and teaches it step by step: each step one visual + one narrated explanation, with her able to interrupt and ask questions at any point.
- Then the viva — framed as “let’s chat about what we did”: 3–5 conversational questions mapped to the lesson’s concepts and graded against the rubric.
- Where she is stuck, the agent *immediately* re-teaches that one concept differently — a new analogy, a diagram, or a curated video in her language (Hindi fallback) — then gently re-checks.
- It closes by updating her progress view (`show_progress`) and telling her what comes next time.

Why this feature second: it is the product’s soul. Onboarding proves the interface; this proves the *mentorship* — personalisation, patience, and the re-teach loop no static e-learning course has.

8.3 Explicitly cut from the MVP

Kiosk hardware; facial login (spoken PIN instead); the marketplace (data model kept, storefront skipped); mock interviews and customer simulations; readiness score (transparent checklist instead); languages beyond 2–3 at demo time; offline mode. Say this list out loud to judges — knowing what you did *not* build is a credibility signal, not a weakness.

8.4 The five-minute demo script

(1) Open the URL, pick Gujarati; the mentor speaks first. (2) Live onboarding: the judge watches the screen rearrange itself as the agent asks about interests — 90 seconds. (3) Cut to a returning user (pre-seeded “Sunita”): greeted by name, lesson resumes. (4) Teach two steps of a lesson, then the viva; deliberately answer one question wrongly and let the agent visibly re-teach with a diagram — this is the applause moment. (5) Show the progress screen and close with the roadmap slide (5 languages, marketplace, kiosk). Rehearse with recorded audio clips as backup in case the venue microphone or network misbehaves.

9. Risks and open questions

- **Latency.** STT + LLM + TTS per turn can reach 3–6 seconds. Mitigate with streaming TTS, a “thinking” earcon (soft chime), and short agent replies. Test early — this is the most likely demo-day pain.
- **Speech recognition on real accents.** Saaras is strong, but test with real Gujarati speakers (not just yourselves) before demo day. Code-mixed replies are the norm, not the edge case.

- **Consent and dignity.** Even in an MVP, ask permission by voice before saving her profile, and offer deletion. With this user population, judges will ask; the DPDP Act 2023 applies.
- **Scope creep.** The brief has ten features; you are shipping two. Every “small addition” before the competition is a threat to the two that must be flawless.
- **Open question for the team:** which skill’s curriculum do we hand-build first? This plan assumes tailoring (Aishi should validate with an NSDC Qualification Pack), but beauty work is an equally good choice — pick one, never both, before the MVP ships.

10. Sources

- Sarvam models overview (Saaras v3, Bulbul v3, Mayura, Sarvam-Translate, Sarvam LLMs): <https://docs.sarvam.ai/api-reference-docs/getting-started/models>
- Bulbul TTS languages and voices: <https://docs.sarvam.ai/api-reference-docs/models/bulbul>
- Building for Indian languages (code-mixing, language codes): <https://docs.sarvam.ai/api-reference-docs/building-for-india>
- Sarvam pricing, free credits, rate limits: <https://docs.sarvam.ai/api-reference-docs/pricing> and <https://docs.sarvam.ai/api-reference-docs/ratelimits>
- Prior art — Vahan, voice-first AI recruiting for blue-collar workers at scale: <https://frontiertech.niti.gov.in/story/voice-ai-at-scale-powering-half-a-million-jobs-a-year-in-india/>
- Prior art — Gram Sootra voice assistant for rural women artisans: <https://www.engineeringforchange.org/projects/voice-recognition-feature-for-rural-textile-producers/>
- LangGraph documentation: <https://langchain-ai.github.io/langgraph/> DSPy: <https://dspai.ai/>
- NSDC Qualification Packs (curriculum grounding): <https://nqr.gov.in/>